

App CAA-TEA — Comunicación Aumentativa y Alternativa para TEA

Aplicación web (PWA) pensada para niños y niñas con Trastorno del Espectro Autista de nivel 2-3 (edad 3-6 años) y para los adultos (familia, terapeutas, profesorado) que les acompañan.

Este README es la guía **completa**: explica qué hace la app, qué necesitas para usarla, cómo instalarla paso a paso, cómo arrancarla, cómo desarrollarla y cómo resolver los problemas más habituales.

Para profundizar en la arquitectura, la API REST completa, el esquema de BD o las decisiones técnicas avanzadas, ver [DESARROLLO.md](#).

Índice

1. [Qué es y qué hace](#)
 2. [Requisitos previos](#)
 3. [Instalación paso a paso](#)
 4. [Configuración \(variables de entorno\)](#)
 5. [Cómo arrancar la app](#)
 6. [Usuarios demo](#)
 7. [Guion de demostración](#)
 8. [Vista del niño \(4 módulos\)](#)
 9. [Sistema de estrellas y niveles](#)
 10. [Vista del adulto \(dashboard clínico\)](#)
 11. [Stack técnico](#)
 12. [Estructura del repositorio](#)
 13. [API REST \(resumen\)](#)
 14. [Desarrollo](#)
 15. [Instalar como PWA en tablet](#)
 16. [Troubleshooting](#)
 17. [Roles del equipo](#)
 18. [Decisiones de diseño](#)
 19. [Preguntas frecuentes](#)
 20. [Licencia y créditos](#)
-

Qué es y qué hace

Las personas con TEA presentan dificultades específicas en **comunicación, interacción social, anticipación de rutinas y regulación emocional**. Esta app cubre esas cuatro áreas con módulos visuales, accesibles y respaldados por pictogramas oficiales de **ARASAAC** (Centro Aragonés de Comunicación Aumentativa y Alternativa, banco de imágenes Creative Commons reconocido en logopedia).

La app tiene **dos vistas independientes**:

Vista	Para quién	Acceso
Niño	El menor que usa la app en una tablet	Toca su nombre en la pantalla inicial
Adulto	Padre/madre/terapeuta/docente	Botón "Soy adulto" + PIN de 4 dígitos (demo: 1234)

Requisitos previos

Requisito	Versión mínima	Notas
Node.js	≥ 18 LTS	Probado con 20.x y 24.x. Descarga desde nodejs.org .
npm	≥ 9	Se instala junto con Node.js.
Navegador	Chrome / Edge actual	Necesario para Web Speech API (síntesis de voz) y PWA.
Conexión a internet	Solo primera vez	Para resolver pictogramas contra la API de ARASAAC. Luego se cachean.

No necesitas PostgreSQL, MySQL, Docker, Python ni Visual Studio Build Tools. La base de datos es SQLite vía `sql.js` (WASM puro). Funciona en Windows, macOS y Linux sin compilación nativa.

Verifica que Node está instalado:

```
node --version      # debe salir v18.x.x o superior
npm --version
```

Instalación paso a paso

1. Descargar el proyecto

```
# Si tienes git:
git clone <url-del-repo>
cd caa-tea-sqlite\caa-tea

# Si tienes un .zip:
# Descomprime y entra en la carpeta caa-tea/
```

2. Instalar dependencias

Hay dos formas equivalentes.

Opción A — Script de workspace (recomendado):

```
npm run install:all
```

Esto instala las dependencias del backend y del frontend en una sola orden.

Opción B — Manual (dos carpetas):

```
cd server
npm install
cd ..

cd client
npm install
cd ..
```

La primera instalación tarda 1-3 minutos según conexión.

3. (Opcional) Configurar variables de entorno

No es obligatorio: si no creas `.env`, la app usa valores por defecto. Solo crea `server/.env` si quieres cambiar el puerto, el secreto JWT o la URL del cliente. Ver [Configuración](#).

4. Listo

No hay paso de migración de BD: el archivo `caa_tea.db` se crea automáticamente la primera vez que arranca el servidor, con los datos de demo precargados.

Configuración (variables de entorno)

Crea (opcionalmente) un archivo `server/.env` con estas variables:

```
PORT=3001
JWT_SECRET=cambia_esto_en_produccion
CLIENT_URL=http://localhost:5173
```

Variable	Por defecto	Para qué sirve
PORT	3001	Puerto del backend Express.
JWT_SECRET	(valor interno)	Firma los JWT de autenticación. Cámbialo en producción.
CLIENT_URL	http://localhost:5173	Origen permitido por CORS. Cambia esto si despliegas el cliente en otro host.

Si trabajas solo en local con los valores por defecto, **puedes saltarte este paso entero.**

Cómo arrancar la app

Necesitas **dos terminales abiertas a la vez**: una para el backend y otra para el frontend.

Terminal 1 — Backend

```
cd server
npm run dev
```

Verás:

```
CAA-TEA API corriendo en http://localhost:3001
Base de datos: caa_tea.db (SQLite via sql.js)
```

```
npm run dev usa nodemon para recargar al guardar cambios. Para producción usa npm
start.
```

Terminal 2 — Frontend

```
cd client
npm run dev
```

Verás algo así:

```
VITE v5.x.x  ready in 500 ms
➔ Local:    http://localhost:5173/
```

Abrir la app

Abre tu navegador en **http://localhost:5173** y aparecerá la pantalla de selección de niño.

Parar la app

Ctrl+C en cada terminal. En Windows, si algún proceso queda colgado:

```
taskkill /F /IM node.exe
```

Construir el frontend para producción

```
cd client
npm run build      # genera client/dist/
npm run preview    # sirve el bundle generado
```

Usuarios demo

Se crean automáticamente al arrancar el servidor por primera vez:

Quién	Cómo entrar	Dato
Niño	Toca el avatar "Mateo"	Sin PIN
Adulto	Botón "Soy adulto" → elige "María (mamá)"	PIN: 1234

Los IDs son **deterministas** (UUIDs fijos en el seed), así que no cambian aunque reinicies el servidor o borres la BD.

Guion de demostración

Mini-guion para enseñar la app a alguien en 2-3 minutos:

1. Abre **http://localhost:5173** → selecciona "Mateo".
2. **Hablar**: cambia entre 2-3 categorías (Comida, Familia, Pedir). Comprueba que cada imagen se corresponde con su etiqueta.
3. **Construye una frase de 4 pictogramas** ("mamá", "por favor", "agua", "gracias") y pulsa "► Escuchar mi frase". El navegador la lee. Mira cómo el contador de estrellas y el nivel suben.
4. **Logros**: muestra la barra de progreso al siguiente nivel y la sección "Cómo ganar estrellas".
5. **Sentir**: elige "Triste" → "Pedir abrazo". Aparece el popup de recompensa.
6. **Mi día**: ve la agenda y completa la primera ranura.
7. **Pulsa el botón flotante Adulto** → confirma → introduce **PIN 1234**.
8. **Dashboard**: muestra las 6 gráficas con datos reales de la sesión que acabas de hacer.
9. **Salir** desde la barra lateral (dirá "↩ Volver con el niño") → la sesión de Mateo se restaura intacta.

Vista del niño (4 módulos)

Cuando el niño selecciona su perfil entra a un panel con **4 botones grandes** en la barra lateral:

Hablar (Constructor de frases)

- 9 categorías temáticas: Comida, Acciones, Sentir, Lugares, Familia, Objetos, Higiene, Tiempo, Pedir.
- ~14 pictogramas por categoría con imagen y etiqueta, en vivo desde ARASAAC.
- Buscador libre (escribir cualquier palabra → aparecen pictogramas relacionados).
- El niño compone una frase tocando hasta 8 pictogramas y pulsa "► Escuchar mi frase". El sistema lee la frase en español usando síntesis de voz nativa del navegador.

Mi día (Anticipación visual)

- Tres ranuras: **AHORA** (actividad en curso), **DESPUÉS** y **LUEGO**.
- Cada ranura muestra el pictograma + etiqueta de la actividad (configurada por el adulto).
- Cuando completa la actividad actual, pulsa " Completar" y la secuencia avanza.

Sentir (Regulador emocional)

- Paso 1: identifica la emoción ahora (Feliz, Triste, Enfadado, Asustado, Calmado, Sorprendido).
- Paso 2: el sistema ofrece **3 estrategias de regulación** específicas a esa emoción (p. ej. "respirar", "pedir un abrazo", "contar hasta 10").
- Si elige una estrategia gana estrellas (autonomía emocional). Si no se ve capaz, hay un botón "Necesito ayuda de un adulto".

Logros

- Muestra el **nivel actual** (1-10) con barra de progreso al siguiente.
- Tabla **"Cómo ganar estrellas"** completamente visible.
- **Racha semanal** real (días consecutivos con actividad).
- **Galería de amigos coleccionables**: cada nivel desbloquea un personaje (Burbujín, Llamita, Estrellux, Aletita, Arcorín, Fungito, Tortulín, Dragoncito, Pulgito, Unicornio).

Botón flotante "Adulto"

En cualquier momento, el niño puede pulsar el botón morado abajo a la derecha para llamar a un adulto. El sistema **guarda la sesión del niño** y muestra la pantalla de PIN. Cuando el adulto termina y pulsa "↩ Volver con el niño", la sesión del niño se restaura intacta sin tener que volver a iniciar sesión.

Sistema de estrellas y niveles

Acción	Estrellas
Frase de 1 pictograma	+2
Frase de 2 o 3 pictogramas	+5
Frase de 4 o más pictogramas	+8
Elegir una estrategia emocional	+10
Completar un paso de la agenda	+15
Bonus de día completo (≥3 acciones distintas)	+20

Niveles 1→10 con curva creciente: **0, 25, 60, 110, 180, 270, 380, 520, 700, 1000** estrellas.

Las estrellas se **persisten en la base de datos por niño**, no en localStorage. Si el niño cambia de tablet o se cierra el navegador, el progreso sigue ahí.

Vista del adulto (dashboard clínico)

Tras introducir PIN, el adulto entra a `/adult/dashboard` y ve **6 gráficas** justificadas teóricamente. Cada una se acompaña de una explicación corta.

1. **Progreso de comunicación** — pictogramas vs palabras por semana.
2. **Uso de la app** — barras diarias / semanales / mensuales con selector.

- 3. **Tipo de comunicación** — frases de 1 pictograma vs frases de 2+ (complejidad lingüística).
- 4. **Errores y tiempo de respuesta** — latencia media + errores (pictos borrados + frases descartadas).
- 5. **Interacción social** — % semanal de pictogramas "sociales" sobre el total.
- 6. **Pictogramas más utilizados** — top 10 con imagen real, etiqueta y frecuencia.

Además, la pestaña **Agenda** permite editar las tres ranuras AHORA / DESPUÉS / LUEGO buscando pictogramas y arrastrándolos a cada slot.

Stack técnico

Capa	Tecnología	Versión
Frontend	React + Vite + PWA	18.3 / 5.4
Estado cliente	Zustand (con <code>persist</code>)	4.5
Routing	React Router DOM	6.24
Gráficas	Recharts	2.12
TTS	Web Speech API (nativa)	—
Backend	Node.js + Express (ESM)	≥18 / 4.19
Base de datos	SQLite vía <code>sql.js</code> (WASM)	1.11
Auth	JWT + bcryptjs	9.0 / 3.0
Pictogramas	API REST de ARASAAC	v1

Importante: usamos `sql.js` en lugar de `better-sqlite3` precisamente para que funcione en **Windows sin necesidad de Python ni Visual Studio Build Tools**.

Estructura del repositorio

```
caa-tea/
├─ README.md           ← este archivo (guía completa)
├─ DESARROLLO.md       ← guía técnica profunda (API, BD, telemetría)
├─ SETUP.md            ← notas extra de instalación en Windows
├─ package.json        ← scripts del workspace
├─ caa_tea.db           ← BD SQLite (se crea automáticamente)
├─ sql/                ← esquema y semilla
│   └─ 001_schema_sqlite.sql
├─ server/              ← backend Express + SQLite
│   └─ index.js
│   └─ db/sqlite.js
│   └─ middleware/auth.js
│   └─ routes/          ← auth, users, phrases, schedule, emotion, dashboard, arasaac, progres
│       └─ utils/        ← levels.js, progress.js
└─ client/              ← frontend React + Vite
    └─ vite.config.js
        └─ src/
            └─ main.jsx
            └─ store.js
            └─ components/
```

```
| | └─ SelectChild.jsx
| | └─ ChildApp.jsx
| | └─ shared/      ← PinGate, RewardPopup
| | └─ modules/     ← PhraseBuilder, Anticipation, EmotionRegulator, Achievements
| | └─ adult/       ← AdultApp (dashboard), ScheduleEditor, VocabularyEditor
└─ hooks/useTTS.js
└─ utils/api.js
```

API REST (resumen)

Base URL: `http://localhost:3001/api` . Las rutas autenticadas llevan `Authorization: Bearer <jwt>` .

Recurso	Endpoints clave
Auth	POST <code>/auth/child-login</code> · POST <code>/auth/adult-login</code>
Users	GET <code>/users/all-children</code> · GET <code>/users/all-adults</code> · GET <code>/users/children</code> · POST <code>/users/event</code>
Phrases	POST <code>/phrases</code> · GET <code>/phrases?childId=&limit=</code>
Schedule	GET <code>/schedule/:childId/today</code> · PUT <code>/schedule/:childId</code> · PATCH <code>/schedule/:childId/advance</code>
Emotion	POST <code>/emotion</code> · GET <code>/emotion?childId=</code>
Dashboard	GET <code>/dashboard/:childId?weeks=8&days=30</code> (6 series + autonomía emocional)
Progress	GET <code>/progress/rules</code> (público) · GET <code>/progress/:childId</code>
ARASAAC	GET <code>/arasaac/search?q=</code> · GET <code>/arasaac/categories</code>
Salud	GET <code>/health</code>

Detalle completo de payloads y respuestas en [DESARROLLO.md](#) .

Desarrollo

Scripts disponibles

Desde la raíz del proyecto (`caa-tea/`):

Comando	Qué hace
<code>npm run install:all</code>	Instala dependencias de server + client.
<code>npm run dev:server</code>	Atajo a <code>cd server && npm run dev</code> .
<code>npm run dev:client</code>	Atajo a <code>cd client && npm run dev</code> .
<code>npm run build:client</code>	Construye el bundle de producción del cliente.

En `server/` :

Comando	Qué hace
<code>npm run dev</code>	Arranca con <code>nodemon</code> (auto-reload).
<code>npm start</code>	Arranca con <code>node</code> plano (para producción).

En `client/` :

Comando	Qué hace
---------	----------

Comando	Qué hace
<code>npm run dev</code>	Vite en modo desarrollo en <code>:5173</code> .
<code>npm run build</code>	Bundle de producción a <code>dist/</code> .
<code>npm run preview</code>	Sirve el bundle ya construido.

Reset de la base de datos

Para volver al estado de fábrica:

```
# Para todos los procesos node (Windows)
taskkill /F /IM node.exe

# Borra la BD (se recreará con datos demo al arrancar)
del caa_tea.db
```

Y opcionalmente limpia la caché de pictogramas en el navegador: DevTools → Application → Local Storage → borra la clave `caa-pictos-v2`.

Añadir un nuevo niño o adulto

Hoy se añade insertando una fila en la tabla `users` con `role='child'` o `role='adult'` (con `pin_hash` bcrypt para adultos). La pantalla de selección recoge a los nuevos usuarios automáticamente. Ya hay tickets abiertos para una pantalla de alta visual desde el dashboard.

Más documentación de desarrollo

Para detalles de:

- Esquema completo de BD (`sql/001_schema_sqlite.sql`).
- Cómo se resuelven los pictogramas dinámicamente.
- Eventos de telemetría disparados por el frontend.
- Cómo extender el sistema de niveles y estrellas.
- Smoke test con `curl`.

Lee [DESARROLLO.md](#).

Instalar como PWA en tablet

1. Asegúrate de que el PC y la tablet están en el **mismo WiFi**.
2. En PowerShell escribe `ipconfig` y anota tu IPv4 (ej: `192.168.1.50`).
3. Edita `server/.env` y pon `CLIENT_URL=http://192.168.1.50:5173` (o usa `*` solo en demo).
4. Reinicia el backend.
5. En Chrome de la tablet ve a `http://192.168.1.50:5173`.
6. Chrome → ⋮ → **Añadir a pantalla de inicio**.

Una vez instalada como PWA: funciona offline parcialmente y cachea las imágenes de ARASAAC.

Troubleshooting

"npm no se reconoce como comando"

Node.js no está instalado o no recargaste la terminal. Cierra y vuelve a abrir VS Code / PowerShell.

"Cannot find package 'sql.js'" o "Cannot find package 'express'"

Faltó el `npm install`:

```
cd server && npm install
cd ../client && npm install
```

Puerto 3001 o 5173 ya en uso

```
# Encuentra el PID que ocupa el puerto:
netstat -ano | findstr :3001
# Mata el proceso (sustituye <PID> por el número):
taskkill /PID <PID> /F
```

La app carga pero no hay usuarios

Borra `caa_tea.db` y reinicia el server. Se recreará con los datos demo.

"PIN incorrecto" con 1234

Pasaba con una versión antigua que sembraba UUID aleatorio para el adulto. Borra `caa_tea.db` y reinicia: el seed actual usa IDs deterministas.

Las imágenes de los pictogramas no aparecen

1. Verifica que el server esté arriba: abre `http://localhost:3001/api/health` → debe responder `{ok:true}`.
2. Comprueba el proxy de Vite: `curl http://localhost:5173/api/arasaac/search?q=comer` debe devolver JSON.
3. Limpia la caché: DevTools → Application → Local Storage → borra `caa-pictos-v2` y recarga.

El dashboard del adulto sale vacío

Es normal hasta que el niño haya creado al menos una frase. Loguéate como Mateo, crea 2-3 frases y vuelve.

Error pidiendo Python o `node-gyp`

No deberías ver esto con la implementación actual (usamos `sql.js`, no `better-sqlite3`). Si lo ves, alguien volvió a `better-sqlite3`. Revisa `server/package.json`.

Roles del equipo

- **Diseño/UX:** paleta de colores, tamaños de pictograma, ergonomía en tablet.
 - **Contenido pedagógico:** ajustar las *keywords* por categoría ([PhraseBuilder.jsx](#)) y las estrategias por emoción ([EmotionRegulator.jsx](#)).
 - **Desarrollo backend:** rutas REST, esquema de BD, fórmulas de progreso.
 - **Desarrollo frontend:** componentes, gráficas del dashboard, accesibilidad.
 - **Demo y memoria:** usar este README + [DESARROLLO.md](#) como base para la presentación.
-

Decisiones de diseño

- **PWA:** la app se puede instalar en una tablet como si fuera nativa, funciona offline parcialmente y cachea las imágenes.
 - **Voz nativa:** Web Speech API del navegador (gratis, en español, sin servidores externos).
 - **Pictogramas dinámicos:** en lugar de hardcodear IDs (que envejecen mal), declaramos solo *keywords* en español y resolvemos los IDs reales contra ARASAAC en el primer uso. Quedan cacheados en el navegador.
 - **SQLite con `sql.js`:** base de datos en archivo único, sin servidor de BD, sin compilación nativa. Funciona en Windows sin Python ni Visual Studio Build Tools.
 - **Sin librería UI:** estilos inline con tokens de color para que el diseño sea fácilmente ajustable.
 - **Tracking ético:** los eventos que registramos (frases, emociones, latencia, errores) se guardan solo en local (en la misma BD del proyecto). No se mandan a ningún tercero.
-

Preguntas frecuentes

¿Necesito conexión a internet? Solo la primera vez que se abre cada categoría (para resolver los pictogramas en ARASAAC). Después se cachean y funciona offline.

¿Y si la API de ARASAAC se cae? Las imágenes ya cacheadas se siguen viendo. El buscador libre dejaría de funcionar hasta que vuelva.

¿El PIN se puede cambiar? Sí, está hashado con bcrypt en `users.pin_hash`. En producción habría una pantalla de gestión; en demo el PIN es 1234 fijo.

¿Qué pasa si dos adultos comparten un mismo niño? La tabla `adult_child_links` permite relación N:M. El dashboard de cada adulto solo muestra a los niños vinculados a él.

¿Cómo añado un nuevo niño? Hoy se añade insertando una fila en la tabla `users` con `role='child'` . La pantalla de selección lo recogerá automáticamente.

¿Puedo desplegar la app en un servidor real? Sí. Construye el cliente con `npm run build` y sirve `client/dist/` desde cualquier static host (Netlify, Vercel, Nginx). El backend lo arrancas con `npm start` en un Node ≥ 18 . Recuerda poner `JWT_SECRET` propio y `CLIENT_URL` apuntando al dominio público.

Licencia y créditos

- **ARASAAC**: pictogramas Creative Commons By-NC-SA. Hay que mantener la atribución cuando se distribuya la app.
- **Web Speech API**: nativa del navegador, sin coste ni servidores.
- **Código del proyecto**: trabajo académico (curso 2025-2026, GIS).